

TRABAJO FIN DE GRADO

Curso 2025/2026

Apuntes

AUTORA: Claudia García-Matarredona Urbano

Puerto Real, 3 de junio de 2026

Índice

1. Investigación previa	3
1.1. ¿Cómo se representa una ecografía?	3
1.2. Aprendizaje sobre Pytorch	3
1.2.1. Construcción de redes convolucionadas	5
1.3. Métricas a considerar	5
2. Jerarquía de fichero	6
3. Desarrollo del preprocesado	7
3.1. Orientar imágenes	7
3.2. Padding	7
3.2.1. Elección de la dimensión máxima	7
3.2.2. Rellenado centrado	7
3.2.3. Implementación final	8
3.3. Aplicación de filtros	8
3.3.1. Gaussiano	8
3.3.2. Mediana	8
3.3.3. Hamming	8
3.3.4. Inverso	8
3.3.5. Transformada de Fourier y regresión lineal	8
3.3.6. Clipping	8
3.4. Otras técnicas	9
3.4.1. Normalización	9
3.4.2. Regularización	9
3.4.3. Penalización	9
3.5. División de datos	9
3.6. Data Augmentation	9
4. Implementación de varias redes	9
4.1. U-Net	9
4.1.1. Pruebas	10
4.2. U-Net++	10
4.2.1. Pruebas	10
4.3. ResNet18	10
4.3.1. Pruebas	11
4.4. ResNet50	12
4.4.1. Pruebas	12
4.5. VGG personalizada	12
4.5.1. Pruebas	12
4.6. VGG clásica	13
4.6.1. Pruebas	13
4.7. Deep Belief Network	13
4.7.1. Pruebas	14
4.8. Transfer Learning	14
4.8.1. Pruebas	14
4.9. SENet	14
4.9.1. Pruebas	14
4.10. DenseNet	15
4.10.1. Pruebas	15
4.11. Multiple Instance Learning (MIL)	15
4.11.1. Pruebas	15

5. Segmentación	16
5.1. U-Net	16
5.2. U-Net++	17
5.3. Aplicación sobre clasificador	17
6. Creación de datos sintéticos	17
7. Desarrollo de una aplicación	18
7.1. Elección del lenguaje	19
7.2. Diseño de la interfaz	19
7.2.1. Tipografía	19
7.2.2. Colores	19
7.2.3. Revisión completa	19
7.3. Implementación en C++	20
7.3.1. Pantalla y estilos	21
7.3.2. Menú	21
7.3.3. Clasificar	22
7.3.4. Segmentar	22
7.3.5. Generar datos	22
7.3.6. Cargar y descargar ecografía	22
7.3.7. Terminal	22
7.4. Pruebas de inferencia	22
7.5. Desarrollo de una web para acceder al proyecto	22
8. Probando diferentes resoluciones	22
9. Comparativa con paper de Leakage	22
10.Problemas	23
10.1. Memoria	23
10.2. Tiempo	23
10.3. Selección del mejor modelo	23
10.4. Testing indeterminista	24
10.5. Concepto de segmentación erróneo	24
11.Resultados	24
12.Bibliografía	26

1. Investigación previa

1.1. ¿Cómo se representa una ecografía?

Las ecografías se guardan en el formato: *.nii.gz*, al cargarlos en *Python* usando *nib.load()* conseguimos un *Nifti1Image*. Esta estructura nos proporciona 3 subestructuras importantes:

- **Datos:** Se consigue mediante la función *get_fdata()* y nos proporciona una matriz $nx \times ny \times nz$ con el tamaño y la intensidad de cada píxel.
- **Header:** Es un atributo de *Nifti1Image* con el mismo nombre, contiene la interpretación de los datos, es decir, el tamaño, orientación y unidades de cada voxel¹.
- **Matriz Affine:** Es otro atributo de *Nifti1Image* y se puede sacar también del *Header*. Consiste de una matriz de 4×4 con: la orientación espacial², posición y tamaño de voxel de cada eje y el *shear*³, un ejemplo podría ser el siguiente:

```
affine = [[ -1.5,  0.0,  0.0,  90.0], # Fila 1: orientación X
          [  0.0,  1.5,  0.0, -126.0], # Fila 2: orientación Y
          [  0.0,  0.0,  1.5, -72.0],  # Fila 3: orientación Z
          [  0.0,  0.0,  0.0,   1.0]]  # Fila 4: Shear
```

Esto significa:

- * Voxel[0,0,0] está en posición (90, -126, -72) en el espacio
- * Cada voxel mide 1.5×1.5×1.5
- * El signo negativo en -1.5 significa que el eje X va de derecha a izquierda

1.2. Aprendizaje sobre Pytorch

Pytorch es una librería que permite crear redes neuronales [1], por lo que su uso será obligatorio para el desarrollo de este proyecto.

Lo primero que hicimos fue instalarlo para su uso en la cpu (para pruebas locales, después usaremos la gpu) con:

```
pip3 install numpy --pre torch torchvision torchaudio ...
--force-reinstall --index-url https://download.pytorch.org/whl/nightly/cpu
```

Después aprendimos sobre los tensores, estos son estructuras de datos (parecidas a los Numpy Array) que pueden hacer operaciones sobre la gpu, por lo que los usaremos para codificar:

- Los inputs de nuestra red
- Las capas internas
- Sus parámetros
- La capa de salida

Volviendo a Pytorch, es importante saber que a la hora de modelar una red neuronal lo que estamos haciendo es crear un modelo de Pytorch, por lo que una red completa sería un conjunto de submodelos Pytorch.

Podemos acceder a estos modelos desde `torch.nn`, siendo los más básicos (hay más [aquí](#)):

¹Un voxel es un píxel de tres dimensiones.

²Existen tres espacios: Axial (va de arriba a abajo, equivale al eje Y), Sagital (va de izquierda a derecha, equivale al eje X) y Coronal (va de frente a atrás, equivale al eje Z).

³El *shear* es una transformación geométrica que desplaza puntos en una dirección proporcional a su coordenada en otra dirección.

- **nn.Module**: clase base para todos los módulos de redes neuronales en Pytorch. Puede definir su propia red neuronal subclasificando Módulo e implementando el método `forward()`.
- **nn.Linear**: esta clase implementa una transformación lineal de los datos de entrada. Toma el tamaño de entrada y el tamaño de salida como parámetros y calcula la salida como una matriz de multiplicación de la entrada y una matriz de peso aprendible, seguida de un término de sesgo opcional.
- **nn.Conv2d**: esta clase implementa una capa convolucional 2D, que se usa comúnmente en tareas de visión por computadora. Toma el tamaño de entrada, el tamaño de salida, el tamaño del núcleo, la zancada, el relleno y la dilatación como parámetros y calcula la salida convolucionando la entrada con un núcleo aprendible.
- **nn.ReLU**: esta clase implementa la función de activación de la unidad lineal rectificada, que se usa comúnmente en las redes neuronales para introducir la no linealidad.

Sabiendo todo esto, ya podríamos crear una red, por ejemplo:

```
from torch import nn

class Net(nn.Module):
    def __init__(self):
        super().__init__() # Hereda de nn
        self.fc1 = nn.Linear(4, 16)
        self.fc2 = nn.Linear(16, 5)
        self.fc3 = nn.Linear(5, 3)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x) # fc = fully connected
        return x
```

Para cargar los datos utilizaríamos un framework cualquiera (Pandas, Polars, etc.) o una ruta concreta. Después podemos hacer una división de datos automática con **train_test_split()** de *Scikit-Learn* o con una función propia y finalmente tendremos que transformar estos conjuntos a tensores.

Es recomendable modularizar todo este proceso en funciones como *DataLoader* o *TensorDataset*.

Para el entrenamiento necesitamos una función de **coste** y un **optimizador**, finalmente sólo habrá que seguir los siguientes pasos:

1. Iniciar la red, el optimizador y la función de coste. La convención es llamar **criterion** a la función de coste, **optim** al optimizador y **net** a la red.
2. Fijar los gradientes a cero aplicando el método **.zero_grad()**. Es importante hacerlo porque Pytorch guarda el buffer de los gradientes internamente.
3. Obtener la predicción de la red, pasando los inputs a nuestra red.
4. Calcular el error generado por la red. Lo podemos hacer con: **criterion(outputs, labels)**.
5. Propagar el error por el resto de la red, lo cual se consigue aplicando el método **backward()** de nuestra función de coste.
6. Aplicar el optimizador para ajustar los parámetros de la red teniendo en cuenta el error que han cometido. Este paso se logra aplicando el método **step()** de nuestro optimizador.
7. Repetir los pasos del 2 al 6 tantas veces como queramos, ya que dichos pasos suponen una iteración.

Para la evaluación del modelo debemos ponerlo en modo **eval**, es decir, hacer: **net.eval()** y después aplicar:

```

with torch.no_grad():

    # Calculate error on test
    test_outputs = net(X_test)
    _, predicted = torch.max(test_outputs.data, 1)
    total_test = y_test.size(0)
    correct_test = (predicted == y_test).sum().item()

```

Para guardar el modelo utilizaremos **torch.save()**. Es **muy** importante que esté en modo **eval**, ya que lo que guardamos es el bias y los pesos (*state_dict()*) generados durante el entrenamiento, no la red en sí mismo.

Para cargarlo utilizaremos el método **.load()**. Lo que hace es cambiar el estado del diccionario al que habíamos guardado, normalmente esto se hace en un modelo limpio, sin entrenar, para ahorrarnos ese tiempo.

1.2.1. Construcción de redes convolucionadas

Aunque el proceso a seguir es el mismo que hemos visto antes, ahora tenemos que usar otras cosas a la hora de cargar las imágenes, porque estas no se pueden guardar en memoria como un **.csv**. En este caso usaremos un *batch*, es decir, un repositorio de donde cargar la imagen, también necesitaremos dos clases ya mencionadas anteriormente para agilizar el proceso:

1. Una clase **Dataset**, que permite cargar los datos en pedazos (`torchvision.datasets.ImageFolder`) y que aplique las transformaciones necesarias. Esta clase debe contar con dos métodos:
 - **__getitem__**: dado un índice, carga una observación, en nuestro caso, una imagen, aplica la transformación que sea necesaria y devuelve tanto la imagen como su label.
 - **__len__**: indica el número de datos que podemos cargar.
2. Un **DataLoader**, el cual utiliza el Dataset definido previamente y nos permite indicar el tamaño de los batch de carga o si las imágenes deben ser aleatorias o no, entre otras cosas.

Por último, para obligar el uso de la gpu a la hora de entrenar la red convolucionada se debe establecer: **net = net.to('cuda')**, aunque también podemos utilizar: **set_default_device()** para que todas las operaciones se hagan en gpu.

Nota: Como el supercomputador trabaja con Linux podemos usar **torch.compile()** para conseguir mejores de rendimiento y tiempo.

1.3. Métricas a considerar

Para la posterior evaluación del modelo, necesitamos métricas que nos informen sobre lo bueno o malo que es, así que en esta sección vamos a explicar las que usaremos y sus valores:

- **Accuracy y Balanced Accuracy:**
 - **Dominio:** $[0, 100]$ (expresado en porcentaje).
 - **Valores óptimos:** Lo más cercano a 100 posible. Un modelo binario “base” suele estar en el 50 %.
 - **Significado:** El Accuracy mide el porcentaje total de predicciones correctas sobre el total de casos.
 - **Diferencia entre ambos:** El Accuracy trata a todas las instancias por igual, mientras que el Balanced Accuracy es la media aritmética del recall de cada clase, evitando que la clase mayoritaria enmascare un mal desempeño en la minoritaria.
- **Precision (Precisión):**
 - **Dominio:** $[0, 1]$
 - **Valores óptimos:** Cercano a 1.

- **Significado:** Representa la calidad de las predicciones positivas: de todo lo que el modelo marcó como positivo, cuántos lo eran realmente.
- **Recall (Sensibilidad) y Specificity (Especificidad):**
 - **Dominio:** $[0, 1]$
 - **Valores óptimos:** Ambos cercanos a 1 para un modelo equilibrado.
 - **Significado:** El Recall mide la capacidad de detectar positivos reales, mientras que la Specificity mide la capacidad de detectar negativos reales.
 - **F1-Score:**
 - **Dominio:** $[0, 1]$
 - **Valores óptimos:** Cercano a 1.
 - **Significado:** Es la media armónica entre Precision y Recall. Se utiliza como métrica única para evaluar modelos con datos desbalanceados.
 - **AUC-ROC:**
 - **Dominio:** $[0.5, 1]$ (0.5 es azar, 1 es discriminación perfecta).
 - **Valores óptimos:** Por encima de 0.8 se considera un buen clasificador.
 - **Significado:** Mide la capacidad del modelo para separar las clases (probabilidad de que el modelo clasifique un ejemplo positivo aleatorio por encima de uno negativo aleatorio).
 - **Loss (Pérdida):**
 - **Dominio:** $[0, \infty)$
 - **Valores óptimos:** Lo más cercano a 0 posible.
 - **Significado:** Representa la “penalización” por predicciones erróneas durante el entrenamiento.
 - **Consideraciones:** Debería de estar por debajo del umbral 0.693 para entropía cruzada binaria.

2. Jerarquía de fichero

El proyecto contiene numerosos ficheros para una mejor modularización, estos serían:

- **crear_tensores.py:** Crea una carpeta de tensores a partir de la base de datos original, aplicando el preprocesado necesario, para agilizar el entrenamiento de las arquitecturas.
- **dividir_datos.py:** Crea una carpeta con los nombres de los ficheros correspondiente a *testing* y *desarrollo*. En *desarrollo* se incluyen diferentes *folds* con datos de *entrenamiento* y *validación*.
- **dataset.py:** Establece un *DataSet* personalizado para acceder y enviar datos al modelo.
- **modelo_x.py:** Establece la arquitectura del modelo a utilizar.
- **labels.csv:** Contiene el nombre de los ficheros y su etiqueta.
- **padding.py:** Contiene las funciones necesarias para aplicar un padding centrado a una ecografía.
- **training.py:** Entrena todos los folds y recoge sus métricas en un fichero *.log*.
- **testing.py:** Testea al modelo con los datos reservados para el *testing* y el mejor modelo obtenido en *training.py* según su *auc-roc*.
- **pipeline_x.py:** Combina todos los ficheros anteriores en un flujo de trabajo.

3. Desarrollo del preprocesado

3.1. Orientar imágenes

Para la orientación de imágenes hemos utilizado la función `nib.as_closest_canonical(imagen)` que devuelve otra imagen cuya orientación cumple el formato RAS (Right-Anterior-Superior).

3.2. Padding

Para el padding, es decir, el rellenado de puntos nulos equitativo para redimensionar todas las imágenes a un tamaño fijo hemos utilizado tres funciones auxiliares:

1. **cargar_eco(ruta)**: Dada una ruta, devuelve los datos, el header y la matriz affine de ficha ecografía.
2. **procesar_directorio(directorio)**: Dado un directorio, carga todas las ecografías usando la función anteriormente explicada en una estructura.
3. **direccionar(ecografia)**: Dada una ecografía utiliza la función descrita en [Subsección 3.1](#) para aplicar el formato RAS.

Ahora ya podemos explicar el proceso de padding que se ha seguido, para ello vamos a realizar varias subsecciones que expliquen qué se ha hecho en cada parte.

3.2.1. Elección de la dimensión máxima

Lo primero que necesitamos son los tamaños máximos de cada eje, ya que si cogemos valores menores habrá ecografías que pierdan información.

Para hacerlo tendremos que ir ecografía a ecografía sacando el tamaño de su matriz `datos`, ya que al representar esta los voxel de la imagen tendrá su dimensión.

Cuando obtengamos el tamaño tendremos que compararlo con las dimensiones máximas almacenadas y sustituir cuando se supere, es decir:

```
dimensiones = [(128, 128, 50), (100, 100, 100)]
# No queremos escoger: (128, 128, 50) porque 128 > 100
# Lo que necesitamos es: (128, 128, 100) - máximo por cada posición
```

Para esta comparativa utilizamos `zip()`, esta función se encarga de emparejar tuplas tal que:

```
a = (1, 2, 3)
b = (4, 5, 6)
list(zip(a, b))
# Resultado: [(1, 4), (2, 5), (3, 6)]
```

Así que lo que haremos será guardar siempre el máximo de las tuplas que nos devuelve `zip()`.

3.2.2. Rellenado centrado

Para esta última parte haremos otra función auxiliar llamada `aplicar_padding(datos, dimension_max)` que dado los datos de una ecografía y la dimensión máxima previamente encontrada usará `np.pad` para añadir ceros según la diferencia entre la dimensión actual y la máxima. El funcionamiento de esta función es la siguiente:

```
arr = np.array([[1, 2],
               [3, 4]])

# Padding: ((arriba, abajo), (izquierda, derecha))
padded = np.pad(arr, ((1, 1), (2, 2)), mode='constant', constant_values=0)

# Resultado:
# [[0 0 0 0 0 0]
```



```
# [0 0 1 2 0 0]
# [0 0 3 4 0 0]
# [0 0 0 0 0 0]]
```

El resultado final se puede ver en la [Figura 1](#).

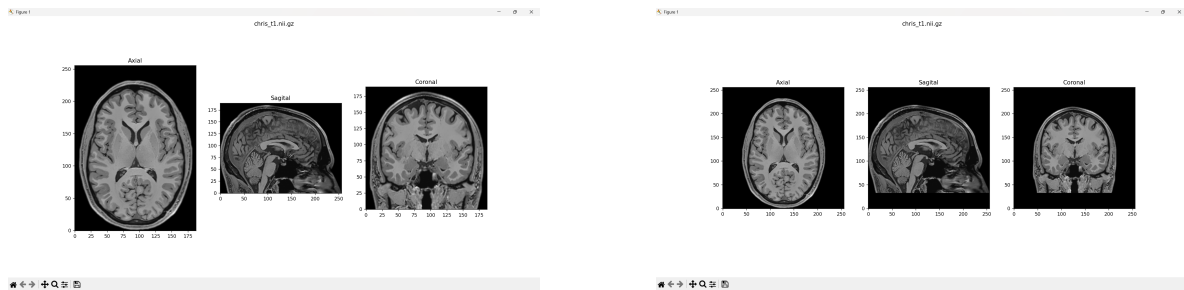


Figura 1: A la izquierda la imagen original y a la derecha con Padding

3.2.3. Implementación final

A la hora de enviar el programa al supercomputador, fue necesario guardar todos los datos en un `.log`, siendo los resultados obtenidos:

- Orientación: { R, A, S }
- Dimensión: $368 \times 368 \times 203$
- Número de ecografías: 4020

Una vez verificada su buen funcionamiento, se tuvo que modularizar para poder usarse con el resto de ficheros.

AL FINAL NO SE USA.

3.3. Aplicación de filtros

3.3.1. Gaussiano

Para reducción de ruido y suavizado general.

3.3.2. Mediana

Para reducir el ruido y cerrar huecos manteniendo la nitidez de los contornos de la imagen.

3.3.3. Hamming

Mejora el contraste al dividir el espectro de frecuencia en bandas parcialmente superpuestas y promediándolas.

3.3.4. Inverso

Amplia el ancho de banda y mejora la resolución axial hasta en un factor de dos.

3.3.5. Transformada de Fourier y regresión lineal

Para derivar parámetros espectrales y ayudar a identificar el tipo de tejido.

3.3.6. Clipping

???

3.4. Otras técnicas

3.4.1. Normalización

Ajuste de intensidad y normalización de canales para que tengan media cero y varianza unitaria.

3.4.2. Regularización

Uso de Drop-out (desactivación aleatoria de neuronas) y normalización por lotes (Batch normalization) para acelerar el entrenamiento y mejorar la estabilidad.

3.4.3. Penalización

Se han probado diferentes técnicas de penalización como la Focal (para bases de datos desbalanceadas) o la BCE (técnica básica).

3.5. División de datos

A la hora de dividir los datos tendremos que incluir pacientes completos en el conjunto *train*, *test* o *val*, ya que sino el modelo podría “aprender” características específicas del paciente en lugar de generalizar.

Para ellos utilizaremos una función dada: *3splits_strict.py* que hace eso mismo manteniendo la proporción de clases.

Esta se encargará de crear una estructura de ficheros con los nombres de cada ecografía para que podamos utilizarlas en el entrenamiento, validación y testeo. El porcentaje de división de los datos ha sido del 80 % para los datos de desarrollo y del 20 % para los de testing.

3.6. Data Augmentation

Para ampliar la base de datos hemos creado 3 copias nuevas de cada imagen, es decir, que guardamos la original y después la copiamos y le aplicamos tres transformaciones por separado:

- Rotación aleatoria de $[-30, 30]^\circ$
- Volteo horizontal
- Clipping del 20 %

Esto ha sido actualizado para intentar crear unas transformaciones más reales, ya que es muy poco probable que se haga una ecografía rotada a 180° , por poner un ejemplo. Ahora haremos 4 copias aparte de la original, siendo estas:

- Rotación aleatoria de $[-30, 30]^\circ$
- Translación aleatoria de $[-10, 10]$ en los ejes x e y
- Clipping del 20 %
- Escalado o zoom aleatorio de $[90, 110] \%$

4. Implementación de varias redes

4.1. U-Net

Hemos decidido empezar por esta red, ya que está especializada en segmentación médica y de hecho se creó para ello [7], por lo que nos puede servir como un buen punto de partida.

Su arquitectura consta de dos partes, un codificador y un decodificador, el primero comprime la imagen y aprende sus características, mientras que el segundo la expande de nuevo a su tamaño original.

En nuestro caso hemos empezado implementando la versión clásica descrita por *Olaf Ronneberger* [7], con 2 convoluciones de 3×3 con *ReLU* [3], seguida de un *MaxPool* en el Encoder y un bloque de *up-conv* y dos convoluciones en el Decoder, siendo la convolución final de 1×1 , véase [Figura 2](#).

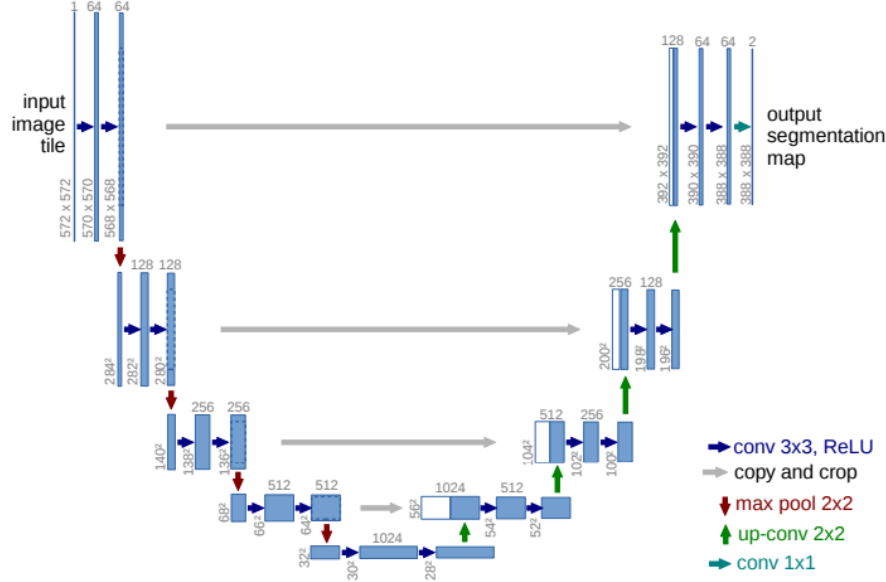


Figura 2: Arquitectura U-Net

Es importante tener en cuenta que será necesario editar estos módulos, ya que el modelo U-Net está pensado para trabajar con imágenes en 2D y nosotros trabajamos con tres dimensiones [11], además de que la arquitectura mostrada en la Figura 2 es de segmentación y nosotros queremos clasificar, por esto, haremos los siguientes cambios:

- Convolución: Aplicaremos el módulo *Conv3d* de *nn*.
- Decoder: En vez de hacer la subida del modelo U, crearemos un *classifier* que transforme el resultado del Encoder en un valor unidimensional.

Para mejorar la convergencia hemos aplicado una normalización del batch [5] y hemos optimizado los módulos 3D con técnicas vistas en el paper de *Fabian Isensee* [6].

4.1.1. Pruebas

Los resultados obtenidos fueron desastrosos, el modelo había aprendido a decir siempre la clase mayoritaria, obteniendo un 78 % de Accuracy, pero un 0 % en el resto de métricas.

4.2. U-Net++

Aunque también se consideró *ViT*, esta se descartó por ser demasiado costosa en tiempo y hemos optado por esta versión mejorada de la *U-Net* (Nested U-Net o U-Net++) [10] que reduce las brechas semántica y por ello trabaja mejor con imágenes médicas.

El principal cambio recae en las *skip connections*, véase Figura 3, así que lo que haremos será establecer estas conexiones anidadas según la siguiente web.

4.2.1. Pruebas

No se llegaron a hacer.

4.3. ResNet18

Al no obtener buenos resultados con los clasificadores anteriores (realmente están pensados para segmentar, así que es normal), hemos decidido aventurarnos con redes completamente convolucionadas

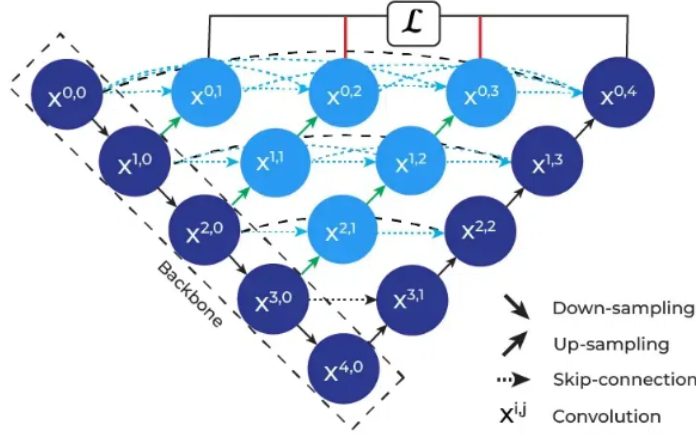


Figura 3: Arquitectura U-Net++

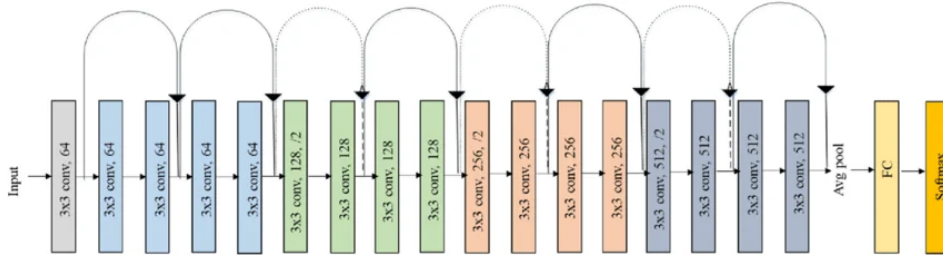


Figura 4: Arquitectura de *ResNet18* clásica

(CNN), especialmente con ResNet18, esta consiste en un codificador de 18 capas de profundidad, véase Figura 4.

Para implementarla sólo hemos tenido que cambiar el fichero `modelo_unet.py` por otro llamado `modelo_resnet18.py` que contendrá un único bloque `nn` de convoluciones y normalizaciones de batch tridimensionales con los tamaño visto en la Figura 4.

4.3.1. Pruebas

Cabe destacar que cada nueva prueba incluye a la anterior, es decir, que la prueba con *Data Augmentation* ya incluye el balanceo de clases.

- **Ejecución primera:** Las métricas obtenidas representan que el modelo no ha aprendido, sino que siempre dice que la imagen es no patológica, de ahí el 75 % de accuracy y 50 % de balanced accuracy. El cero en f1-score refuerza esto y un AUC_ROC por debajo del 0.5 también indica que el modelo responde de forma “aleatoria” o sesgada.
- **Ejecución con balanceo de clases:** Ha habido una leve mejoría. Ahora el modelo se arriesga más, lo que significa que ha aprendido un mínimo, es decir, hemos reducido el sesgo.
- **Ejecución con Data Augmentation:** Ha habido una mejoría bastante notable. Ahora el modelo sabe diferenciar mucho mejor entre clases, aunque es cierto que tiene un porcentaje de acierto demasiado bajo, tendremos que intentar aumentarlo aplicando filtros o con otra red.
- **Ejecución con filtros previos:** Los resultados han empeorado.
- **Ejecución con toda la base de datos + Focal + DA:** Los resultados han sido los mejores hasta la fecha, véase la Tabla 1.

Métrica	Valor
Loss	0.0325
Accuracy	0.8562
Balanced Accuracy	0.8625
Precision	0.6591
Recall	0.8750
Specificity	0.8500
F1-Score	0.7519
AUC-ROC	0.9380

Cuadro 1: Resultados de evaluación del modelo

4.4. ResNet50

Hemos implementado una versión de la ResNet algo más profunda (ResNet50 [4]) para ver cómo se desempeña. Aparte de ser más profunda, esta arquitectura utiliza bloques “Bottleneck” (tres convoluciones: 1×1 , 3×3 y 1×1), en vez del “BasicBlock” (dos convoluciones de 3×3) de ResNet18, para reducir la carga computacional y permitir una mayor profundidad. Véase la Figura 5 para una mejor visión de cómo es la arquitectura.

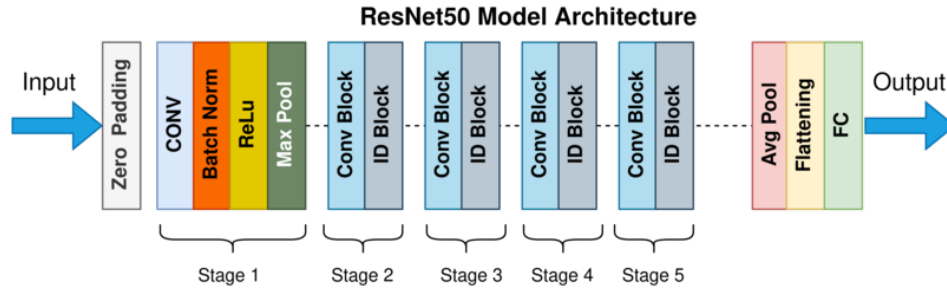


Figura 5: Arquitectura ResNet50

4.4.1. Pruebas

Como este modelo es más profundo y tarda más, pero no ofrece ninguna mejoría con respecto a su versión menos profunda, hemos decidido dejarlo.

4.5. VGG personalizada

Basándonos en un paper publicado en *ultrasound in obstetrics & gynecology* [9], hemos implementado la red VGG de la Figura 6.

4.5.1. Pruebas

Hemos repetido las pruebas con las mismas bases de datos y ficheros de etiquetado que en la Subsección 4.3:

- **Ejecución primera:** Pasa algo muy parecido a la primera ejecución de la ResNet18.
- **Ejecución con balanceo de clases:** Ha habido una leve mejoría, pero sigue sin aprender, aunque ahora es consciente de que no sabe diferenciar muy bien.
- **Ejecución con Data Augmentation:** Las métricas no han sufrido ninguna mejora, así que vamos a abandonar este modelo e implementar una VGG clásica.

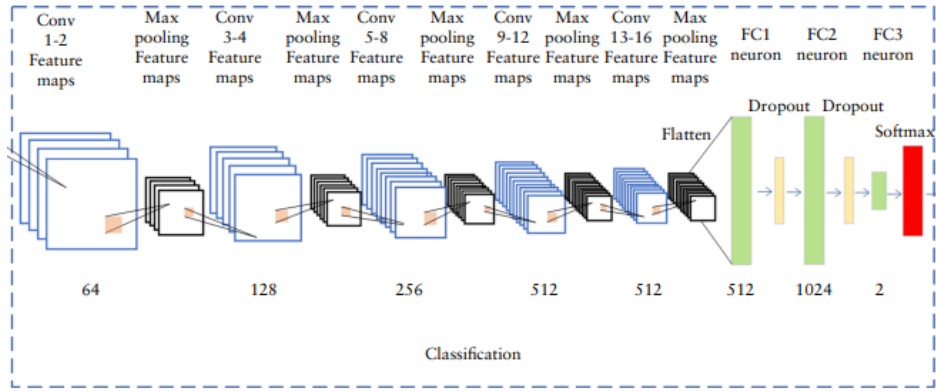


Figura 6: VGG del paper

4.6. VGG clásica

La VGG clásica [8] consta de muchas convoluciones de 3×3 , véase Figura 7.

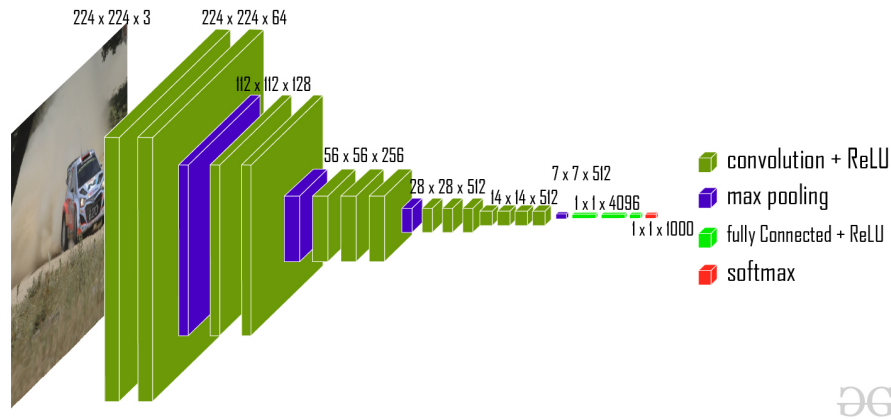


Figura 7: Vgg16 clásica

Simplemente hemos editado el código de la vgg personalizada para que cumpla con esta nueva arquitectura y la hemos sometido a las pruebas más recientes que ya se ha visto que dan buenos resultados en la ResNet18.

4.6.1. Pruebas

- **Ejecución con Data Augmentation:** No da malos resultados, aunque son peores que con la ResNet18.
- **Ejecución con filtros previos:** Los resultados son mucho peores, como el modelo tarda mucho en entrenarse y realmente no aporta mejoras con respecto a la ResNet18, hemos decidido abandonar la arquitectura vgg, tampoco implementaremos la vgg19, ya que hemos leído que suele estancarse al costarle más converger.

4.7. Deep Belief Network

Tras varias lecturas hemos optado por probar esta red que pretende aprender de forma no supervisada las características de las imágenes, teniendo incluso la capacidad de reproducirla (se podría usar para crear nuevas ecografías, junto con las redes GAN). Al realizar un preentrenamiento de las capas podemos empezar el entrenamiento con unos pesos ya orientados, en vez de aleatorios.

4.7.1. Pruebas

Primero hicimos un preentrenamiento en el que sólo congelábamos las capas, es decir, que no había aprendizaje. Después implementamos un auto-encoder para el entreno previo, y aunque los resultados no han sido muy buenos, es cierto que mejoran.

Como último recurso, hemos decidido probar la ejecución con toda la base de datos + Focal + DA, pero los resultados no han sido buenos. En resumen, hemos obtenido un acierto balanceado del 48 % y un auc_roc de 0.53, habiendo rutas más prometedoras vemos un desperdicio de tiempo seguir invirtiendo en esta arquitectura.

4.8. Transfer Learning

Viendo el problema de los datos hemos decidido acudir a redes preentrenadas con imágenes médicas tridimensionales a ver si mejoran el rendimiento de nuestra mejor arquitectura hasta el momento, ResNet18. Para ello hemos utilizado el repositorio de MedicalNet [2] disponible tanto en [GitHub](#) como en [HuggingFace](#). De este último hemos descargado los pesos, los cuál cargaremos con una función personalizada que adaptará dichos valores a nuestra red ResNet18 clásica.

4.8.1. Pruebas

- **Ejecución con Data Augmentation:** Ha dado peores resultados que la versión entrenada desde cero, con un auc_roc del 54 %, probablemente debido a la falta de imágenes o a sesgos aprendidos en el preentreno.
- **Ejecución con toda la base de datos + Focal + DA:** Los resultados han mejorado en gran medida, véase [Tabla 2](#), pero siguen sin superar a la versión entrenada desde cero, por esta razón y por la carencia de modelos preentrenados con imágenes médicas en 3D, hemos decidido abandonar esta opción.

Métrica	Valor
Loss	0.0579
Accuracy	0.8388
Balanced Accuracy	0.7580
Precision	0.7093
Recall	0.5970
Specificity	0.9189
F1-Score	0.6483
AUC-ROC	0.8652

Cuadro 2: Resultados del Testing del Modelo

4.9. SENet

También conocida como *squeeze-and-excitation network*, esta arquitectura se suele usar como apoyo de otras, en nuestro caso usaremos la ResNet18, ya que ha sido la que mejores resultados nos ha dado en todas las pruebas.

El objetivo es añadir un bloque que permita a la arquitectura base entender las interdependencias de los datos, en resumen, el bloque realizará una compresión de los datos para después excitarlos y obtener pesos más precisos que resalten información relevante.

Así conseguiremos reducir ruido y que la red sea más precisa, podemos tomar el funcionamiento de la U-Net como apoyo para entender todo lo explicado anteriormente.

4.9.1. Pruebas

- **Ejecución con toda la base de datos + Focal + DA:** Los resultados han sido sorprendentes, mejorando en la mayoría de métricas, a excepción del acierto balanceado, véase la [Tabla 3](#) para observar las métricas.

Métrica	Valor
Loss	0.0526
Accuracy	0.9029
Balanced Accuracy	0.8475
Precision	0.8529
Recall	0.7371
Specificity	0.9579
F1-Score	0.7908
AUC-ROC	0.9429

Cuadro 3: Resultados del Testing del Modelo

4.10. DenseNet

Esta arquitectura es muy similar a la ResNet, con la diferencia de que en vez de sumar las características, es decir, pasar los pesos de forma secuencial, las concatenaremos, por lo que deberíamos obtener una reutilización de características más eficiente. Véase la [Figura 8](#) para más información.

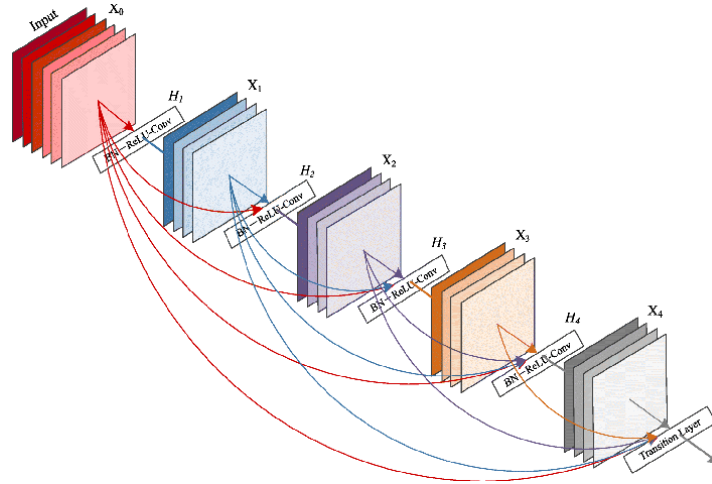


Figura 8: Arquitectura DenseNet

4.10.1. Pruebas

- **Ejecución con toda la base de datos + Focal + DA:** No se ha llegado a hacer individualmente, sino con la ejecución de 5 semillas, véase [Tabla 11](#).
- **Ejecución con SENet:** El f1-score ha salido nulo, aunque el auc_roc ha sido muy bueno, 0.93. Por desgracia tarda demasiado y es peor que la ResNet18, así que hemos decidido dejarla.

4.11. Multiple Instance Learning (MIL)

Como último intento de mejora, hemos incluido un bloque de aprendizaje de instancias múltiples, es decir, que la clasificación va en función a un grupo y no a una instancia, al modelo ResNet18 con bloque SENet. Lo que haremos será dividir cada ecografía en una serie de parches, estos serán el grupo, y decidir en función a ellos si esa ecografía es más probable de ser patológica o no.

4.11.1. Pruebas

- **Ejecución con toda la base de datos + Focal + DA:** No se ha llegado a hacer individualmente, sino con la ejecución de 5 semillas, véase [Tabla 11](#).

5. Segmentación

Tras probar todas estas arquitecturas con diferentes preprocesados, hemos decidido implementar una segmentación previa para reducir el ruido, quedándonos solo con la parte importante de la ecografía para clasificarla. Para ellos, hemos implementado una función que pasará todas las máscaras segmentadas a tensores, acelerando el proceso de entrenamiento.

Antes de seguir, es importante tener en cuenta que ahora usaremos otras métricas propias de la segmentación:

- **Loss (Pérdida - Dice Loss):**

- **Dominio:** $[0, 1]$.
- **Valores óptimos:** Lo más cercano a 0 posible.
- **Significado:** Mide la discrepancia entre la máscara predicha y la real basándose en el solapamiento de áreas.
- **Consideraciones:** Se calcula como 1-Dice Score. Un valor de 0 indica un solapamiento perfecto.

- **Dice Score (F1-Score para segmentación):**

- **Dominio:** $[0, 1]$.
- **Valores óptimos:** Cercano a 1.
- **Significado:** Mide la similitud estadística entre dos muestras. Se define como el doble de la intersección dividido por la suma de las áreas de la predicción y la etiqueta real.

- **IoU (Intersection over Union / Jaccard Index):**

- **Dominio:** $[0, 1]$.
- **Valores óptimos:** Por encima de 0,7 se considera una segmentación muy sólida.
- **Significado:** Mide el área de intersección entre la predicción y la máscara real, dividida por el área de su unión. Es una métrica más estricta y penalizadora que el Dice Score.

- **Recall (Sensibilidad):**

- **Dominio:** $[0, 1]$
- **Valores óptimos:** Cercano a 1
- **Significado:** En segmentación, representa la fracción de píxeles/vóxeles de la máscara real que fueron correctamente identificados por el modelo.

Ahora que tenemos todo esto claro, vamos a entrar en las arquitecturas utilizadas en la segmentación.

5.1. U-Net

Hemos implementado la misma red que en la [Subsección 4.1](#), con la particularidad de que esta vez sí hemos diseñado el *Decoder*. Los resultados de la segmentación pueden verse en la [Tabla 4](#). Cabe destacar que estas métricas son el resultado de:

1. Seleccionar una semilla del conjunto $[42, 99, 123, 777, 2026]$.
2. Particionar la base de datos en *train*, *val* y *test*.
3. Entrenar con cada fold y seleccionar el mejor para el testeo, en función a la métrica *dice*.
4. Guardar los resultados del test.
5. Repetir con todas las semillas y crear las métricas finales como la media de todos los tests.

Al igual que hacemos en la [Sección 11](#) y con la *Nested-U-Net*.

Métrica	Valor
Loss	0.1987
Dice Score	0.9365
IoU Jaccard	0.8972
Precision	0.9708
Recall Sensitivity	0.9258

Cuadro 4: Resultados de la segmentación con U-Net

5.2. U-Net++

Al igual que antes, hemos hecho lo mismo que en la [Subsección 4.2](#). Los resultados de la segmentación pueden verse en la [Tabla 5](#).

Métrica	Valor
Loss	0.1964
Dice Score	0.9408
IoU Jaccard	0.9052
Precision	0.9679
Recall Sensitivity	0.9368

Cuadro 5: Resultados de la segmentación con Nested U-Net

5.3. Aplicación sobre clasificador

Ahora vamos a comprobar si vale la pena una segmentación previa a la clasificación, para ello vamos a repetir el entrenamiento de la ResNet18 con el bloque SENet, pero habiendo multiplicado entre sí los tensores de las ecografías originales con sus máscaras. De esta forma aislaremos la zona importante del ultrasonido, dejando todo el ruido de fondo a cero. Los resultados de esta prueba pueden verse en la [Tabla 6](#).

Métrica	Valor
Loss	0.2378
Accuracy	0.7974
Balanced Accuracy	0.5422
Precision	0.4090
Recall	0.1150
Specificity	0.9694
F1-Score	0.1538
AUC-ROC	0.6993

Cuadro 6: Resultados de segmentación previa

Aquí nos preguntamos, ¿por qué han salido métricas tan bajas? Así que hicimos un script para convertir los tensores en ecografías y otro que las visualizara, véase los resultados en la [Figura 9](#).

Como se puede ver, la segmentación se había hecho de forma inversa, por lo solo tuvimos que invertir los valores de la máscara y ahora nuestra segmentación ya aislaba el cráneo, véase la [Figura 10](#).

Al repetir el entrenamiento hemos obtenido las métricas vistas en la [Tabla 7](#).

6. Creación de datos sintéticos

Para generar nuevas ecografías hemos decidido acudir a las redes GAN, estas son una red neuronal compuesta por dos partes: el generador, que se encarga de crear las imágenes, y el discriminador, que se encarga de decidir si la imagen es original o no. Véase la [Figura 11](#) para una mayor aclaración.

Para implemetarla, debemos duplicar todo lo que hemos hecho hasta ahora, pues cada parte de la red GAN se entrena de forma “independiente”. Lo metemos entre comillas, porque realmente ambas

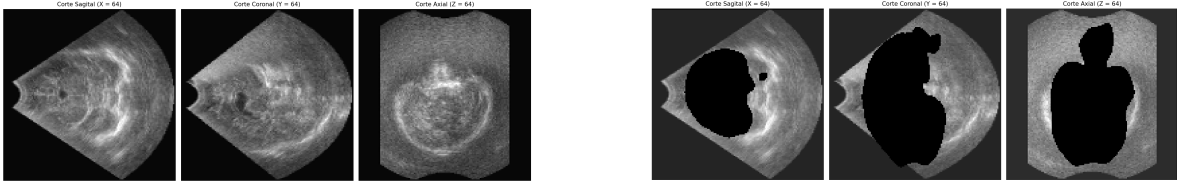


Figura 9: A la izquierda la imagen original y a la derecha la segmentación tras multiplicar la máscara

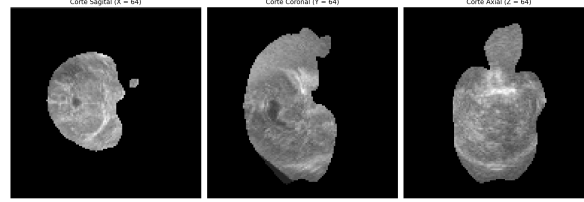


Figura 10: Segmentación correcta

redes están combatiendo constantemente para derrotar a la otra, pero cada una tiene su optimizador, su gradiente, etc.

Además, aquí no hace falta un conjunto de validación, ya que es un entrenamiento no supervisado que se basa en crear imágenes, no de reconocerlas. Por ello, combinaremos los conjuntos *train* y *val* generados por nuestro fichero de división de datos y después haremos una validación con FID en 3D, es decir, Medical-FID. Medical-FID es una métrica adaptada para representar la diferencia del vector de características, generado por un clasificador de 3 dimensiones, entre la distribución estadística de un conjunto completo de imágenes reales frente a la distribución de un conjunto de imágenes generadas, es decir, la siguiente fórmula:

$$d^2 = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Si nos devuelve un número cercano a cero, podemos suponer que el conjunto de imágenes generada es realista. Nosotros utilizaremos la ResNet18 con el bloque SENet para evaluar la red GAN, véase la TABLA para conocer las métricas obtenidas.

Cabe destacar que hemos obligado a que el número de *epoch* sea igual a la longitud del conjunto de test, además, la longitud del vector de ruido inicial de la red GAN es de 256.

7. Desarrollo de una aplicación

Para implementar todas las técnicas vistas, vamos a desarrollar una aplicación instalable capaz de segmentar el volumen craneal, clasificar ecografías y generar datos sintéticos.

Métrica	Valor
Loss	0.1123
Accuracy	0.8463
Balanced Accuracy	0.7009
Precision	0.7283
Recall	0.4499
Specificity	0.9518
F1-Score	0.5142
AUC-ROC	0.8375

Cuadro 7: Resultados de segmentación previa buena

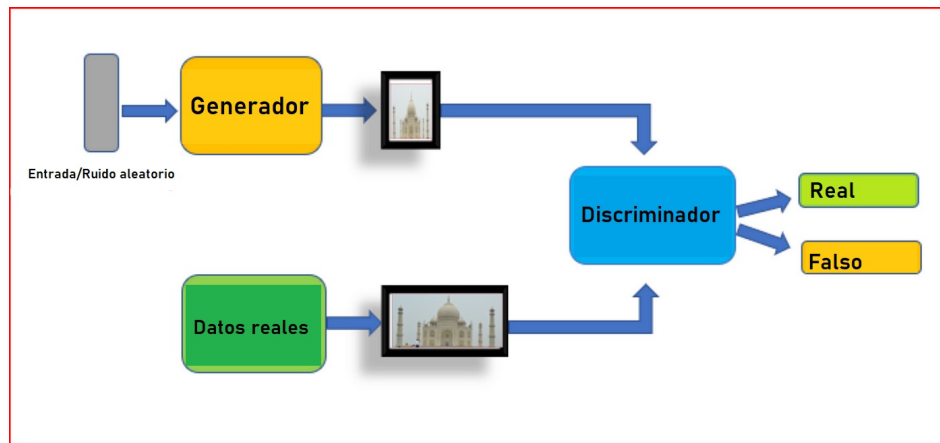


Figura 11: Arquitectura de la red GAN

7.1. Elección del lenguaje

Hasta ahora, hemos desarrollado todo el proyecto en *Python*, pero esto no tiene por qué ser así para la aplicación, por lo que vamos a hacer una breve ponderación de las mejores opciones:

- **Python (Nativo + PyInstaller):** Permite reutilizar el código de PyTorch al 100 %. La inferencia es directa, pero requiere empaquetar el entorno completo en un ejecutable único (.exe/.app) mediante PyInstaller para que el usuario final no tenga que instalar dependencias ni lidiar con entornos de ejecución.
- **C# / .NET (Vía ONNX Runtime):** Excelente para entornos Windows. El modelo de PyTorch se exporta a formato .onnx usando `torch.onnx.export`, lo que agiliza drásticamente la inferencia. Para el usuario es ideal, ya que permite crear instaladores nativos tradicionales (asistentes “Siguiente-Siguiente”) muy ligeros y profesionales.
- **C++ (LibTorch o ONNX):** Ofrece la inferencia más rápida, eficiente y con menor consumo de memoria. El usuario final recibe un programa ultra-ligero y de instalación inmediata que no requiere de Python, facilitando su ejecución incluso en ordenadores con hardware limitado.

Dado que la velocidad en la inferencia es muy importante, porque son modelos pesados, y que tenemos más experiencia con *C++*, hemos elegido esta opción.

7.2. Diseño de la interfaz

La idea es hacer la menor cantidad de desplegados posible, teniendo una única interfaz amplia y clara. Para ello, hemos realizado un boceto a modo de guía en *Canva*, véase la [Figura 12](#).

Y le hemos pedido a *Gemini* que lo revise para obtener un efecto visual más agradable, médico y andaluz.

7.2.1. Tipografía

Para los títulos y la mayoría de textos hemos elegido **Inter**, ya que tiene letras grandes, claras y profesionales, mientras que texto de soporte, como la terminal, usaremos **IBM Plex Mono**, ya que tiene ese efecto técnico.

7.2.2. Colores

Hemos incluido todos los colores propuestos y su uso en una tabla, véase la [Tabla 8](#).

7.2.3. Revisión completa

Finalmente, nuestro nuevo boceto, ya revisado con el diseño propuesto, es el de la [Figura 13](#).

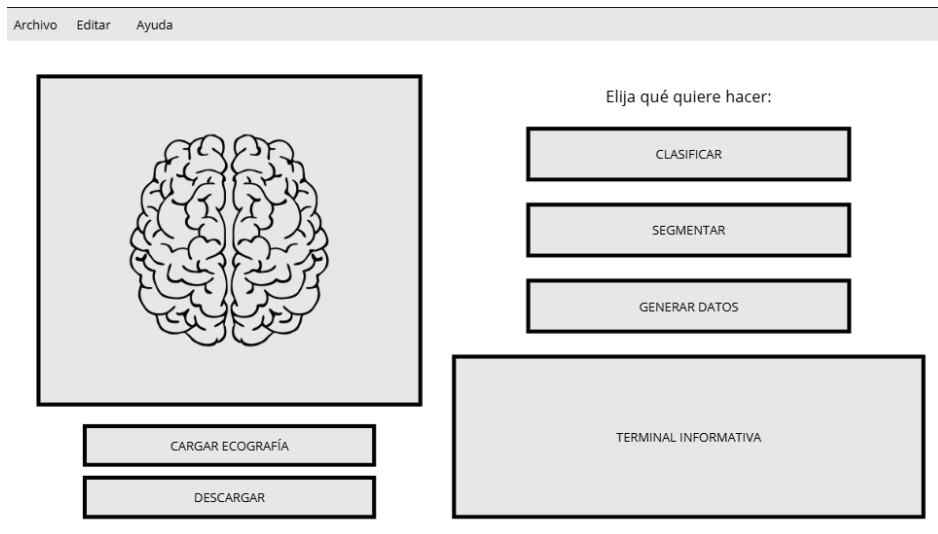


Figura 12: Diseño inicial de la aplicación

Uso / Componente	Nombre del Color	Muestra	Código HEX
<i>Fondos y Superficies</i>			
Fondo de la Ventana	Blanco Suave		#FDFDFD
Tarjetas y Paneles	Blanco Puro		#FFFFFF
Texto Principal	Gris Oscuro		#212121
Texto de Soporte (Terminal)	Gris Medio		#424242
<i>Colores de Énfasis (Identidad)</i>			
Botón Principal / Enfoque	Verde Clínico Profundo		#2A8C7A
Botones Secundarios (Hover)	Verde Menta Suave		#D1EFD9
Detalles / Iconografía	Azul Sanitario		#B2EBF2
<i>Estados de la Aplicación</i>			
Alertas Críticas / Errores	Rojo Médico		#EF5350
Advertencias / Cargas	Naranja Aviso		#FFB300

Cuadro 8: Paleta de colores del Sistema de Diseño

7.3. Implementación en C++

Usaremos **Qt Creator** para el diseño de la interfaz (funciona con *Widgets*), **OpenCV** para pintar las ecografías y **ONNX** para la gestión de los modelos.

Fue necesario obtener una licencia de estudiante para usar Qt, ya que es un software de pago. Pero una vez conseguido, simplemente abrimos el programa y creamos tres ficheros:

- **main.cpp**: Este se encarga de mostrar la pantalla principal y reconocer las acciones que ocurren durante su ejecución.
- **mainwindow.cpp**: Aquí desarrollaremos la interfaz de la pantalla principal, definiendo funciones para:
 - Cargar modelos
 - Aplicar estilos
 - Pintar ecografías
 - Etc.
- **mainwindow.h**: Declaramos los atributos y métodos de la clase anterior.

Ahora es cuestión de copiar el diseño propuesto y añadir funcionalidad a los botones.

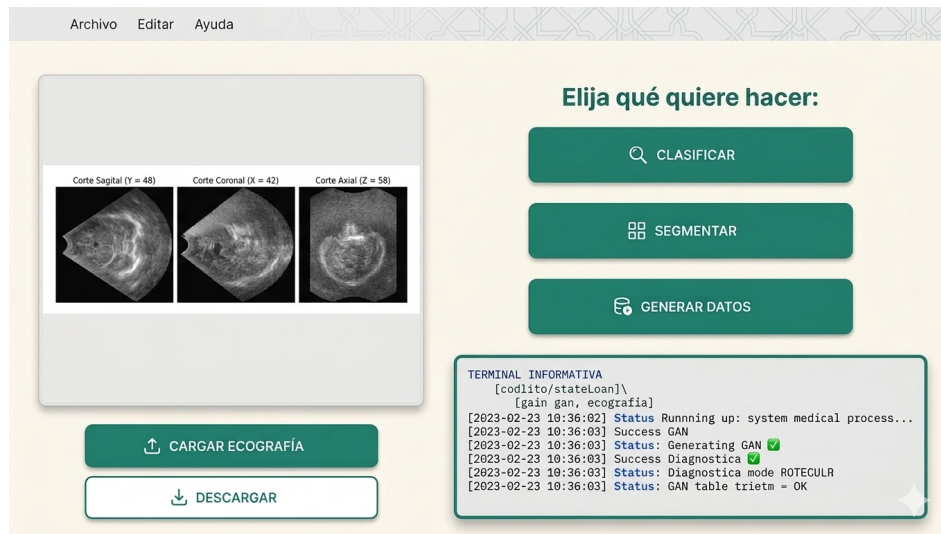


Figura 13: Boceto revisado por *Gemini*

7.3.1. Pantalla y estilos

Lo primero ha sido dividir la pantalla en dos columnas, la izquierda (gestión de imágenes) y la derecha (acciones y terminal). Una vez hecho esto, hemos aplicado los estilos ya explicados, véase la [Figura 14](#).

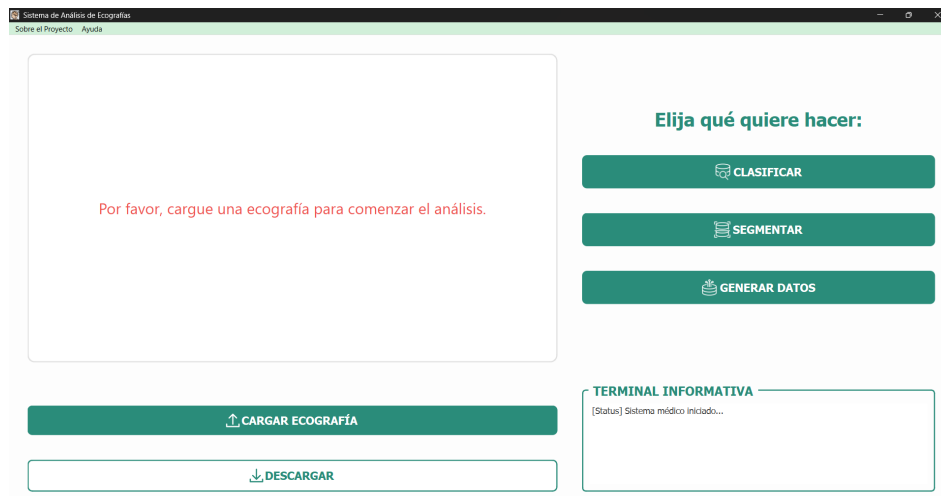


Figura 14: Diseño final de la aplicación con Qt

7.3.2. Menú

Aquí tenemos dos botones, *Sobre el Proyecto* que nos redirigirá a una web con toda la documentación y *Ayuda*, que consta de dos acciones, véase la [Figura 15](#).

Ambas muestran un mensaje por pantalla, la primera explicando la funcionalidad de los botones y la segunda estableciendo tanto el *Copyright* del proyecto, como los derechos de autor de los artistas que han desarrollado los iconos y logos utilizados, véase la [Figura 16](#).

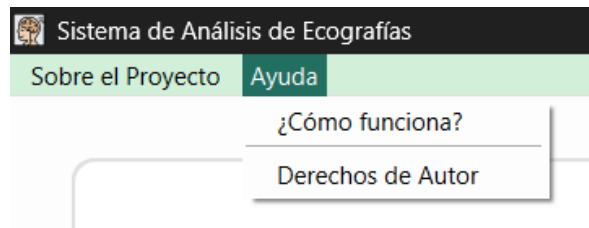


Figura 15: Menú de barra

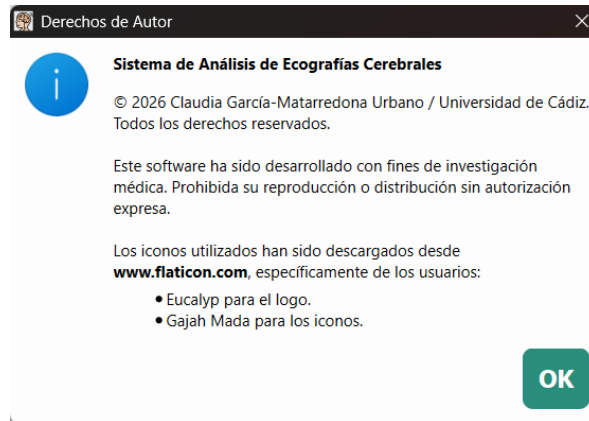


Figura 16: Mensaje de *Derechos de Autor*

7.3.3. Clasificar

7.3.4. Segmentar

7.3.5. Generar datos

7.3.6. Cargar y descargar ecografía

7.3.7. Terminal

7.4. Pruebas de inferencia

7.5. Desarrollo de una web para acceder al proyecto

Para que los usuarios puedan informarse sobre el proyecto, ver la documentación del mismo y descargarse la aplicación, hemos desarrollado una pequeña web. Esta repite las opciones de diseño ya descritas anteriormente y solo constará de una pantalla.

8. Probando diferentes resoluciones

Un factor que hemos omitido hasta ahora, pero que es de gran importancia, es la resolución. Hasta ahora hemos estado entrenando y evaluando los modelos con tensores de (128, 128, 128), pero no hemos probado con otros tamaños, por lo que en esta sección haremos una comparativa con el modelo ResNet18, ya que es el más rápido, y tensores de diferentes dimensiones, véase la [Tabla 9](#) para observar los resultados.

9. Comparativa con paper de Leakage

Hemos editado la ResNet18 clásica que hemos estado utilizando para compararla con los resultados de un paper local no publicado. Para hacerlo hemos hecho los siguientes cambios:

- Cambiar el optimizador *Adam* por *AdamW*.

Métrica	64	96	128
Loss	0.0599	0.0602	0.0738
Accuracy (%)	86.5860	85.9679	86.9633
Balanced Accuracy (%)	74.5505	79.8699	76.0676
Precision	0.7821	0.7055	0.7720
Recall	0.5362	0.6845	0.5692
Specificity	0.9548	0.9129	0.9522
F1-Score	0.6033	0.6516	0.6293
AUC-ROC	0.8913	0.8962	0.8891

Cuadro 9: Comparativa de métricas por resolución

- Usar la base de datos limpia, sin Data Augmentation.
- Quitar el early stopping.
- Añadir la métrica PR-AUC.

En la [Tabla 10](#), hemos recogido las métricas necesarias. Cabe destacar que los datos originales sufrían de *leakage*, es decir, que los conjuntos *train*, *val* y *test* compartían ecografías de un mismo paciente. Esto genera un sobreajuste, ya que el modelo realmente está reconociendo al paciente y no la ecografía. Además, nuestras ejecuciones han sido con 5 semillas, mientras que el paper original sólo utilizaba 3.

Model	ROC-AUC (95 % CI)	PR-AUC	Bal. Acc.	Sens.	Spec.
Original	0.90 [0.81–0.95]	0.81	0.73	0.49	0.97
Mío	0.89 [0.82–0.95]	0.76	0.75	0.54	0.96
Con DA	0.90 [0.83–0.97]	0.78	0.74	0.51	0.98

Cuadro 10: Comparación de métricas paper

10. Problemas

10.1. Memoria

El principal problema que tuvimos fue la memoria, ya que al principio intentábamos cargar todos los datos de las imágenes en la RAM y esto era inviable por su enorme tamaño, así que aplicamos una carga individual, es decir, bajo demanda.

10.2. Tiempo

Nuestro flujo de trabajo inicial tardaba demasiado (unas 13 horas por fold), así que decidimos implementar una función que crease una carpeta local con los tensores de cada imagen, así el entrenamiento tardaría mucho menos (menos de una hora), al no tener que preprocesar cada imagen de forma individual.

Además, implementamos limpiezas de caché, variables, uso de herramientas de *memory allocation* y otras técnicas para aumentar la rapidez de ejecución.

10.3. Selección del mejor modelo

Durante gran parte de las pruebas hemos estado usando la métrica de *auc_roc* para decidir el mejor modelo, pero hacíamos las comparaciones usando la versión antigua que compara la pérdida, por lo que no se elegía correctamente ya que la lógica estaba invertida.

10.4. Testing indeterminista

Hasta bastante avanzado el proyecto se estaba haciendo el testing con el modelo en modo entrenamiento, en vez de en modo evaluación, por lo que obteníamos resultados no del todo fiables. Por lo que tuvimos que repetir todos los experimentos finales para poder hacer una comparativa real.

10.5. Concepto de segmentación erróneo

A la hora de empezar con la segmentación hicimos un planteamiento erróneo en el que pretendíamos obtener unas máscaras con los vóxeles utilizados en la clasificación. Pero lo que realmente teníamos que hacer era aislar el volumen cerebral en vez de encontrar las anomalías, así que tuvimos que reentrenar las redes.

11. Resultados

Finalmente, y para poder recoger los resultados de forma homogénea, hemos ejecutado todas las arquitecturas propuestas 5 veces (con diferentes semillas, concretamente serían la 42, 99, 123, 777 y 2026, para obtener una media) con la opción: **Ejecución con toda la base de datos + Focal + DA**, habiendo editado el Data Augmentation con respecto a la [Sección 4](#), véase la [Subsección 3.6](#), y poniendo el optimizador *AdamW*, en vez de *Adam*. Las métricas obtenidas están disponibles en la [Tabla 11](#).

Métrica	RN18	RN50	RN18(SE)	RN18(TL)	RN18(SE+MIL)
Loss	0.095	0.147	0.104	0.240	2.035
Accuracy	86.663	83.195	87.071	64.158	85.309
Bal. Acc.	74.824	67.116	76.255	68.112	72.597
Precision	0.749	0.692	0.759	0.375	0.676
Recall	0.544	0.395	0.582	0.738	0.514
Specificity	0.952	0.947	0.943	0.624	0.938
F1-Score	0.610	0.455	0.630	0.459	0.580
AUC-ROC	0.881	0.829	0.882	0.742	0.819

Métrica	DN	DBN	VGG paper	VGG16
Loss	0.205	0.069	0.060	0.060
Accuracy	81.598	63.228	79.924	79.924
Bal. Acc.	60.417	54.093	50.000	50.000
Precision	0.655	0.245	0.000	0.000
Recall	0.245	0.378	0.000	0.000
Specificity	0.964	0.704	1.000	1.000
F1-Score	0.266	0.279	0.000	0.000
AUC-ROC	0.879	0.553	0.510	0.485

Cuadro 11: Comparación de métricas medias

Además, hemos repetido la prueba con la ResNet18, pero con la anterior base de datos (data augmentation inicial), para ver cómo varía con respecto a la nueva, véase la [Tabla 12](#).

Métrica	ResNet18 (DA actualizada)	ResNet18 (DA inicial)
Loss	0.095	0.075
Accuracy	86.663	87.845
Balanced Accuracy	74.824	75.505
Precision	0.749	0.748
Recall	0.544	0.557
Specificity	0.952	0.954
F1-Score	0.610	0.625
AUC-ROC	0.881	0.883

Cuadro 12: Comparación de bases de datos en ResNet18

12. Bibliografía

Referencias

- [1] Pytorch tutorial. <https://anderfernandez.com/blog/tutorial-de-pytorch-2/>.
- [2] Sihong Chen, Kai Ma, and Yefeng Zheng. Med3d: Transfer learning for 3d medical image analysis. *arXiv preprint arXiv:1904.00625*, 2019.
- [3] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In Geoffrey Gordon, David Dunson, and Miroslav Dudík, editors, *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 315–323, Fort Lauderdale, FL, USA, 11–13 Apr 2011. PMLR. URL: <https://proceedings.mlr.press/v15/glorot11a.html>.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL: <https://arxiv.org/abs/1512.03385>, [arXiv:1512.03385](#).
- [5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015. URL: <https://arxiv.org/abs/1502.03167>, [arXiv:1502.03167](#).
- [6] Fabian Isensee, Jens Petersen, Andre Klein, David Zimmerer, Paul F. Jaeger, Simon Kohl, Jakob Wasserthal, Gregor Koehler, Tobias Norajitra, Sebastian Wirkert, and Klaus H. Maier-Hein. nnunet: Self-adapting framework for u-net-based medical image segmentation, 2018. URL: <https://arxiv.org/abs/1809.10486>, [arXiv:1809.10486](#).
- [7] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL: <https://arxiv.org/abs/1505.04597>, [arXiv:1505.04597](#).
- [8] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015. URL: <https://arxiv.org/abs/1409.1556>, [arXiv:1409.1556](#).
- [9] H. N. Xie, N. Wang, M. He, L. H. Zhang, H. M. Cai, J. B. Xian, M. F. Lin, J. Zheng, and Y. Z. Yang. Using deep-learning algorithms to classify fetal brain ultrasound images as normal or abnormal. *ULTRASOUND IN OBSTETRICS & GYNECOLOGY*, 2020. URL: <https://obgyn.onlinelibrary.wiley.com/doi/10.1002/uog.21967>.
- [10] Zongwei Zhou, Md Mahfuzur Rahman Siddiquee, Nima Tajbakhsh, and Jianming Liang. Unet++: A nested u-net architecture for medical image segmentation. In Danail Stoyanov, Zeike Taylor, Gustavo Carneiro, Tanveer Syeda-Mahmood, Anne Martel, Lena Maier-Hein, João Manuel R.S. Tavares, Andrew Bradley, João Paulo Papa, Vasileios Belagiannis, Jacinto C. Nascimento, Zhi Lu, Sailesh Conjeti, Mehdi Moradi, Hayit Greenspan, and Anant Madabhushi, editors, *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, pages 3–11, Cham, 2018. Springer International Publishing.
- [11] Özgün Çiçek, Ahmed Abdulkadir, Soeren S. Lienkamp, Thomas Brox, and Olaf Ronneberger. 3d u-net: Learning dense volumetric segmentation from sparse annotation, 2016. URL: <https://arxiv.org/abs/1606.06650>, [arXiv:1606.06650](#).